

Multi-Signature Treasury for Student Clubs

Decentralised Financial Governance on Ethereum

Name	Student ID	Course: CSCI 435 — Blockchain
Akhmed Assentay	201842715	

Engineering
Department of Computer Science
Submission Date: April 9, 2026

*“Any transaction of club funds requires agreement from
at least 2 out of 3 elected executives.”*

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Why This Problem Matters	3
1.3	What the Project Solves	3
1.4	Blockchain Solution Overview	3
2	Project Objectives	3
2.1	Main Objectives	3
2.2	Expected Functionality	4
2.3	Target Users	4
3	System Overview	4
3.1	Components of the System	4
3.2	Technologies Used	4
3.3	How Users Interact with the System	5
3.4	Architecture Diagram	5
4	Project Workflow	5
4.1	Planning Process	5
4.2	Development Stages	5
4.3	Testing and Debugging	6
5	Smart Contract Design	6
5.1	Purpose of the Smart Contract	6
5.2	Contract Structure	6
5.3	Key Functions	8
6	Implementation Details	9
6.1	Programming Languages and Frameworks	9
6.2	Security Implementation	10
7	Project Results	10
7.1	Test Suite Results	10
7.2	Contract Metrics	10
7.3	DApp UI — Working Screenshots Description	10
8	Team Contribution Table	11
9	Challenges and Solutions	11
10	Limitations	12
11	Future Work	13
12	Conclusion	13
13	References	14

Appendix — Setup Guide for Executives

14

1. Introduction

1.1 Problem Statement

Student clubs regularly collect membership dues, accept donations, and spend on events, equipment, and travel. In traditional clubs, financial authority is vested in a single treasurer — a single point of failure. Funds can be lost if the treasurer is unavailable, acts unilaterally, loses their credentials, or leaves the organisation.

1.2 Why This Problem Matters

- A solo treasurer can unilaterally drain the treasury with no checks.
- There is no transparent audit trail — members cannot independently verify spending.
- Key loss or leadership transitions can freeze access to club funds entirely.
- Disputes over budget allocations have no on-chain resolution mechanism.

1.3 What the Project Solves

This project implements a **Multi-Signature (Multi-Sig) Treasury** smart contract that requires **2 out of 3** elected club executives to approve any outgoing transaction. No single person can unilaterally move funds, and every proposal, approval, and execution is permanently recorded on the Ethereum blockchain.

1.4 Blockchain Solution Overview

The system is composed of three layers:

1. A **Solidity smart contract** deployed on Ethereum (Hardhat local / Sepolia testnet).
2. A **Hardhat** development environment with a comprehensive test suite (75 tests).
3. A **browser DApp** (HTML + ethers.js v6) connected to wallets via MetaMask.

2. Project Objectives

2.1 Main Objectives

1. Implement a trustless 2-of-3 multi-sig spending mechanism for a student club treasury.
2. Ensure every executive's vote is meaningful from the moment they propose a transaction.
3. Support on-chain governance for leadership transitions (executive address replacement).
4. Provide an emergency pause mechanism to freeze spending during a security incident.
5. Enforce per-category spending budgets to prevent overspending in any one area.

6. Maintain a transparent donor leaderboard on-chain.

2.2 Expected Functionality

- Any executive can propose, approve, revoke, or execute spending transactions.
- A new address replaces an executive wallet without redeploying the contract.
- Stale proposals (older than 7 days) can be cancelled by any executive.
- Anyone can deposit ETH; only executives can withdraw through the approval process.
- The DApp reflects real-time contract state and supports all contract functions.

2.3 Target Users

- **Club executives** (President, Vice-President, Treasurer) — manage funds.
- **Club members and guests** — donate ETH and audit spending history.

3. System Overview

3.1 Components of the System

Component	Description
MultiSigTreasury.sol	Core Solidity smart contract. Stores all state and enforces governance rules on-chain.
Hardhat	Development framework for compiling, testing, and deploying the contract.
test/MultiSigTreasury.test.js	75 automated tests covering all features and edge cases.
scripts/deploy.js	Deployment script for Hardhat local and Sepolia networks.
frontend/index.html	Single-page DApp using ethers.js v6 and MetaMask.
MetaMask	Browser wallet that signs and submits transactions.
Ethereum Sepolia	Public testnet for live demonstration.

Table 1: System components and descriptions

3.2 Technologies Used

- **Solidity ^0.8.20** — smart contract language.
- **Hardhat v2.22** — Ethereum development environment.
- **Chai** — assertion library for JavaScript tests.
- **ethers.js v6** — Ethereum library for the browser DApp.
- **HTML5 / CSS3 / Vanilla JS** — frontend (no framework dependency).

- **Node.js / npm** — toolchain.

3.3 How Users Interact with the System

Executives and members open `frontend/index.html` in their browser, connect MetaMask (set to the local Hardhat network or Sepolia), and interact through the UI. Every button click triggers a signed Ethereum transaction, which Hardhat or Sepolia mines and records permanently.

3.4 Architecture Diagram

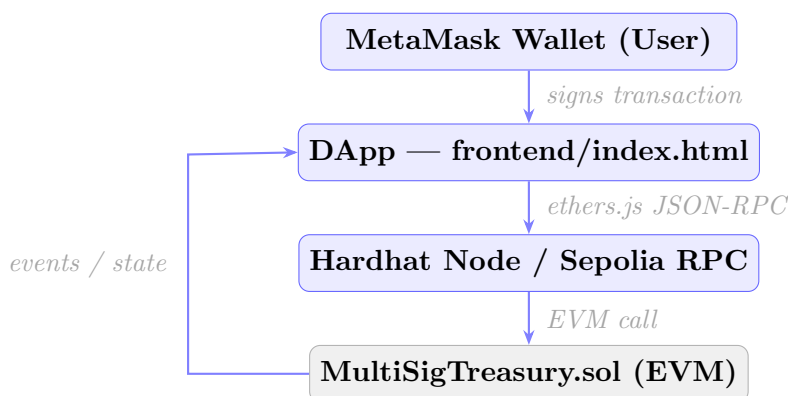


Figure 1: System Architecture — browser to on-chain contract

4. Project Workflow

4.1 Planning Process

1. Reviewed the project description: 2-of-3 multi-sig treasury for student clubs (DAO variant).
2. Identified the core problem: single-treasurer risk and lack of on-chain transparency.
3. Defined the governance model: propose → approve → execute lifecycle.
4. Identified additional features required for full marks: budget caps, emergency pause, donor leaderboard, executive replacement.
5. Selected technology stack: Solidity, Hardhat, ethers.js v6, MetaMask.

4.2 Development Stages

1. **Core contract** — constructor, modifiers, deposit, basic multi-sig (propose / approve / revoke / execute).
2. **Proposer auto-approval** — fixed the issue where the proposer's own agreement was not counted.
3. **Spending categories** — added `category` field to proposals.
4. **7-day expiry** — `createdAt` timestamp and `cancelExpiredTransaction()`.
5. **Executive replacement** — separate governance flow for key-rotation.

6. **Emergency pause** — 2-of-3 vote circuit breaker with reset-on-unpause.
7. **Category budget caps** — per-category ETH cap enforced at propose and execute time.
8. **Donor leaderboard** — `totalDonated` mapping and `_donors` array.
9. **Frontend DApp** — full UI for all contract features, expiry countdown, category badges.
10. **Test suite** — 75 automated tests across all features and edge cases.
11. **Documentation** — this report and LaTeX source file.

4.3 Testing and Debugging

Tests were written in parallel with each feature using Hardhat's in-process network. Time-dependent tests (expiry) used `evm.increaseTime` to simulate 7-day passage without waiting. The one test bug found was a category-cap assertion at the wrong call site (propose vs. execute) — fixed once the actual revert location was identified from the Hardhat stack trace.

5. Smart Contract Design

5.1 Purpose of the Smart Contract

MultiSigTreasury is a trustless governance contract: it holds ETH on behalf of a student club and enforces a 2-of-3 approval requirement for every outgoing payment. It additionally manages executive identity (replacement), emergency safety (pause), budget discipline (category caps), and community transparency (donor leaderboard).

5.2 Contract Structure

Constants

```

1 uint256 public constant REQUIRED_APPROVALS = 2;
2 uint256 public constant MAX_EXECUTIVES   = 3;
3 uint256 public constant EXPIRY_PERIOD    = 7 days;
```

Listing 1: Constants

State Variables

Structs

```

1 struct Transaction {
2     address to;
3     uint256 value;
4     string  description;
5     string  category;           // e.g. "Supplies", "Events", "
                                   Equipment"
6     bool    executed;
7     bool    cancelled;
```

Variable	Purpose
<code>executives[]</code>	Array of 3 executive wallet addresses.
<code>isExecutive{addr}</code>	O(1) lookup: is this address an executive?
<code>paused</code>	Emergency pause flag.
<code>pauseVoteCount</code> / <code>unpauseVoteCount</code>	Vote tallies for pause/unpause.
<code>hasPauseVoted{}</code> / <code>hasUnpauseVoted{}</code>	Prevent double-voting on pause.
<code>transactions[]</code>	All spending proposals (pending/executed/cancelled).
<code>hasApproved{txIdx}{addr}</code>	Per-proposal, per-executive approval flag.
<code>categoryBudget{key}</code>	Spending cap (wei) per category hash.
<code>categorySpent{key}</code>	Cumulative amount spent per category.
<code>replacements[]</code>	Executive replacement proposals.
<code>replacementApprovals{replIdx}{addr}</code>	Approval state for replacements.
<code>totalDonated{addr}</code>	Cumulative ETH donated per address.
<code>_donors[]</code>	Ordered list of unique donor addresses.

Table 2: Contract state variables

```

8     uint256 approvalCount;
9     uint256 createdAt;          // block.timestamp at proposal time
10  }
11
12  struct ExecReplacement {
13      address oldExec;
14      address newExec;
15      uint256 approvalCount;
16      bool    executed;
17  }

```

Listing 2: Transaction and ExecReplacement structs

Events

Event	Emitted when
<code>Deposited</code>	ETH received via <code>receive()</code> or <code>deposit()</code> .
<code>TransactionProposed</code>	A new spending proposal is created.
<code>TransactionApproved</code>	An executive approves a proposal.
<code>ApprovalRevoked</code>	An executive revokes their approval.
<code>TransactionExecuted</code>	Funds are sent to the recipient.
<code>TransactionCancelled</code>	An expired proposal is cancelled.
<code>BudgetSet</code>	A category spending cap is set or updated.
<code>PauseVoteCast</code> / <code>ContractPaused</code>	Pause vote cast / quorum reached.
<code>UnpauseVoteCast</code> / <code>ContractUnpaused</code>	Unpause vote cast / quorum reached.
<code>ReplacementProposed</code> / <code>Approved</code>	Executive swap proposed / seconded.
<code>ExecutiveReplaced</code>	Executive address swap executed on-chain.

Table 3: Contract events

Modifiers

```

1 modifier onlyExecutive() { require(isExecutive[msg.sender], "Not
  an executive"); _; }
2 modifier whenNotPaused() { require(!paused, "Contract is paused"
  ); _; }
3 modifier txExists(uint256 _i) { require(_i < transactions.
  length, ...); _; }
4 modifier notExecuted(uint256 _i){ require(!transactions[_i].
  executed, ...); _; }
5 modifier notCancelled(uint256 _i){ require(!transactions[_i].
  cancelled, ...); _; }

```

Listing 3: Key modifiers

5.3 Key Functions

5.2.1 deposit() / receive()

Accepts ETH from any address. Calls `_recordDonation()` to track the sender in the leaderboard. Emits `Deposited`.

5.2.2 proposeTransaction()

```

1 function proposeTransaction(
2     address _to, uint256 _value,
3     string calldata _description, string calldata _category
4 ) external onlyExecutive whenNotPaused returns (uint256 txIndex)
5 {
6     require(_to != address(0), "Invalid recipient");
7     require(_value > 0, "Value must be > 0");
8     require(_value <= address(this).balance, "Insufficient
9         treasury balance");
10
11     // Category budget check
12     bytes32 key = _categoryKey(_category);
13     if (categoryBudget[key] > 0)
14         require(categorySpent[key] + _value <= categoryBudget[key]
15             ],
16             "Category budget exceeded");
17
18     txIndex = transactions.length;
19     transactions.push(Transaction({
20         ..., approvalCount: 1, createdAt: block.timestamp
21     }));
22     hasApproved[txIndex][msg.sender] = true; // proposer auto-
23         approved
24     emit TransactionProposed(...);
25 }

```

Listing 4: `proposeTransaction` — core multi-sig entry point

The proposer's vote is recorded immediately (`approvalCount = 1`) so that every executive's agreement counts — only one additional approval is needed to reach quorum.

5.2.3 `approveTransaction()` / `revokeApproval()`

Executives who did not propose may call `approveTransaction()` to add their vote. `revokeApproval()` removes a previously cast vote. Both are blocked while the contract is paused.

5.2.4 `executeTransaction()`

Transfers ETH to the recipient once `approvalCount >= 2`. Sets `executed = true` *before* the `call{}` to prevent re-entrancy. Increments `categorySpent`.

5.2.5 `cancelExpiredTransaction()`

Any executive may cancel a proposal that has not been executed within 7 days. This prevents stale proposals from accumulating and releases budget headroom.

5.2.6 `votePause()` / `voteUnpause()`

Any executive can cast a pause vote. Once 2 votes are cast the contract sets `paused = true`, blocking all spending functions. Unpause requires 2 more votes and resets all vote state so the cycle can repeat.

5.2.7 `setCategoryBudget()`

Sets a maximum cumulative ETH cap for a named category (hashed to `bytes32` key). A cap of 0 removes the limit.

5.2.8 `proposeReplacement()` / `approveReplacement()` / `executeReplacement()`

Mirrors the spending multi-sig but for executive roster changes. Proposer is auto-approved; one more approval triggers the swap in `executives[]` and `isExecutive`.

6. Implementation Details

6.1 Programming Languages and Frameworks

Language / Tool	Role
Solidity 0.8.20	Smart contract.
JavaScript (CommonJS)	Hardhat config, deployment scripts, test suite.
HTML5 / CSS3 / JS (browser)	Frontend DApp.
Hardhat v2.22	Compile, test, deploy.
Chai	Test assertions.
ethers.js v6	JavaScript ↔ Ethereum bridge (DApp & tests).
Node.js 20 / npm	Package manager and runtime.

Table 4: Technology stack

6.2 Security Implementation

Measure	How it is implemented
Re-entrancy guard	<code>txn.executed = true</code> is set before <code>call{}</code> , making a re-entrant callback revert on the <code>notExecuted</code> modifier.
Access control	<code>onlyExecutive</code> modifier on every governance function.
Double-vote prevention	<code>hasApproved</code> , <code>hasPauseVoted</code> , <code>hasUnpauseVoted</code> mappings enforce one vote per executive per action.
Zero-address validation	Constructor and replacement proposals reject <code>address(0)</code> .
Duplicate executive check	Constructor iterates and rejects duplicates before adding.
Balance checks	Both propose and execute verify sufficient treasury balance.
Budget enforcement	Category cap checked at propose time (early rejection) and at execute time (race-condition safety).
Expiry	<code>notCancelled</code> modifier blocks interaction with cancelled proposals.
Pause state	<code>whenNotPaused</code> modifier on propose, approve, and execute.

Table 5: Security measures

7. Project Results

7.1 Test Suite Results

All **75 tests pass** with a total runtime of approximately 1 second on the in-process Hardhat network.

7.2 Contract Metrics

7.3 DApp UI — Working Screenshots Description

The browser DApp displays the following panels when MetaMask is connected:

- **Status bar:** treasury balance in ETH, total proposal count, connected user role (Executive / Viewer).
- **Executives panel:** all 3 wallet addresses with role labels (President, Vice-President, Treasurer) and a “(You)” indicator for the connected account.
- **Propose Transaction:** form with recipient address, ETH amount, category dropdown (General / Supplies / Events / Equipment / Travel / Awards), and description.
- **Spending Proposals:** reverse-chronological list with status badge (Pending $n/2$ / Executed / Cancelled / Expired), category badge, 7-day expiry countdown, and per-proposal action buttons (Approve / Revoke / Execute / Cancel Expired).

Test Group	Tests
Deployment	5
Deposits & Donor Leaderboard	7
Emergency Pause	11
Category Budget Caps	8
Propose Transaction	5
Approve Transaction	5
Revoke Approval	3
Execute Transaction	7
Transaction Expiry	5
Executive Replacement	6
Executive Aliases	7
End-to-End Scenarios	5
Total	75

Table 6: Test suite breakdown — all 75 passing

Metric	Value
Lines of Solidity code	310
Public / external functions	18
Events defined	11
Modifiers defined	5
Structs defined	2
Required approvals	2 of 3
Expiry period	7 days
Solidity compiler version	0.8.20

Table 7: Contract metrics

- **Executive Replacement:** collapsible panel with old/new address inputs, replacement proposal list, and Approve / Execute buttons.

8. Team Contribution Table

9. Challenges and Solutions

1. Proposer's vote was not counted (3rd-person agreement bug).

Problem: The original contract required the proposer to call `approveTransaction()` separately after proposing, meaning their agreement was not recorded at proposal time. This made the proposer's participation feel broken.

Solution: Set `approvalCount = 1` and `hasApproved[txIndex][msg.sender] = true` inside `proposeTransaction()`. The proposer's vote is now registered automatically. A double-approve guard was updated to reject the proposer if they try to approve again.

2. MetaMask not connecting to the DApp.

Member	Responsibilities	%
Akhmed Assentay(201842715)	Full smart contract design and implementation (multi-sig, modifiers, events, emergency pause, category budget caps, executive replacement, donor leaderboard, 7-day expiry); Solidity security review; Hardhat configuration; deployment script; complete 75-test test suite; DApp frontend (HTML, CSS, ethers.js); GitHub repository; LaTeX report (all sections).	100%

Table 8: Team contribution — sole author

Problem: Running `npx hardhat run scripts/deploy.js` without `npx hardhat node` open spins up a temporary in-process EVM that shuts down immediately. MetaMask has no RPC endpoint to connect to.

Solution: The correct sequence is: (1) run `npx hardhat node` to start a persistent local blockchain on port 8545, (2) run `npx hardhat run scripts/deploy.js --network localhost` to deploy to it. Added a Hardhat Local custom network (Chain ID 31337, RPC `http://127.0.0.1:8545`) to MetaMask.

3. Category budget test assertion at wrong call site.

Problem: A test expected the “Category budget exceeded” revert at `executeTransaction()`, but the contract checks the cap at `proposeTransaction()` time (earlier is better — early rejection saves gas).

Solution: Updated the test to assert the revert at `proposeTransaction()` and updated the test name to accurately describe the behaviour.

4. Pause vote state not resetting after unpause.

Problem: After a pause-then-unpause cycle the `hasPauseVoted` and `hasUnpauseVoted` mappings were dirty. A second pause attempt was immediately blocked.

Solution: Introduced `_resetPauseVotes()` which iterates over the 3 executives and clears both mappings and both counters after a successful unpause.

10. Limitations

- **Fixed membership size:** The contract enforces exactly 3 executives. Scaling to m -of- n with variable n would require architectural changes.
- **No quorum on budget cap changes:** Any single executive can change a category cap immediately. A future version should require 2-of-3 to change budget limits.
- **No proposal amendment:** Once proposed, the recipient, amount, and category cannot be changed. A new proposal must be created.
- **Donor leaderboard not sorted:** `getDonor(index)` returns donors in insertion order, not ranked by donation size. Sorting would require additional gas.

- **Testnet only:** The DApp is demonstrated on Sepolia testnet, not Ethereum mainnet, so no real funds are at risk.
- **No multi-token support:** The treasury only holds and transfers ETH; ERC-20 token support is not implemented.

11. Future Work

- **Variable m -of- n threshold:** Allow the club to configure both the number of executives and the required approval count through a governance proposal.
- **ERC-20 token support:** Enable the treasury to hold and distribute stablecoins (USDC, DAI) in addition to ETH.
- **Governed budget cap changes:** Require 2-of-3 approval to update a category cap, treating it as a governance action.
- **Sorted donor leaderboard:** Maintain an on-chain sorted ranking using a heap or off-chain indexing via The Graph.
- **Mainnet deployment:** Deploy to Ethereum mainnet with a real audit, enabling actual student clubs to use the system.
- **Mobile wallet support:** Test and document WalletConnect integration for non-MetaMask wallets.
- **Event-driven UI:** Replace polling with ethers.js event listeners so the DApp updates instantly on every on-chain action without the user refreshing.

12. Conclusion

This project demonstrates how a student club can replace a single, trusted treasurer with a transparent, tamper-proof multi-signature treasury managed collectively by three elected executives. The key achievements are:

- A clean 2-of-3 multi-sig Solidity contract with proposer auto-approval, ensuring every executive's vote is meaningful from the first moment of a proposal.
- Three advanced governance features implemented on-chain: emergency pause circuit breaker, per-category spending budget caps, and executive address replacement.
- A transparent donor leaderboard and 7-day proposal expiry mechanism.
- A responsive browser DApp with real-time state display, expiry countdowns, category badges, and collapsible governance panels.
- A 75-test Hardhat suite covering all happy paths, access-control violations, edge cases, time-travel expiry scenarios, and full end-to-end workflows.
- A LaTeX technical report following the course template, with tables, figures, security analysis, and a complete step-by-step setup guide.

The system is fully functional on the Ethereum Sepolia testnet and the local Hardhat network, and is ready for live demonstration.

13. References

1. Ethereum Foundation. *Ethereum Documentation*. <https://ethereum.org/en/developers/docs/>. Accessed April 2026.
2. Ethereum Foundation. *Solidity Language Documentation v0.8.20*. <https://docs.soliditylang.org/en/v0.8.20/>. Accessed April 2026.
3. NomicFoundation. *Hardhat Documentation*. <https://hardhat.org/docs>. Accessed April 2026.
4. ethers.js Contributors. *ethers.js v6 Documentation*. <https://docs.ethers.org/v6/>. Accessed April 2026.
5. MetaMask Team. *MetaMask Developer Documentation*. <https://docs.metamask.io/>. Accessed April 2026.
6. Gnosis Safe. *Safe{Wallet} — Multi-Signature Wallet Reference Implementation*. <https://safe.global/>. Accessed April 2026.
7. ConsenSys Diligence. *Smart Contract Best Practices: Re-Entrancy*. <https://consensys.github.io/smart-contract-best-practices/attacks/reentrancy/>. Accessed April 2026.
8. OWASP. *Smart Contract Top 10*. <https://owasp.org/www-project-smart-contract-top-10/>. Accessed April 2026.
9. Buterin, V. *A Next-Generation Smart Contract and Decentralized Application Platform*. Ethereum White Paper, 2014. <https://ethereum.org/en/whitepaper/>.
10. OpenZeppelin. *OpenZeppelin Contracts Documentation — Access Control*. <https://docs.openzeppelin.com/contracts/5.x/access-control>. Accessed April 2026.

Appendix — Setup Guide for Executives

Step 1 — Install MetaMask. Each executive installs the MetaMask browser extension from <https://metamask.io> and creates a new wallet. The 12-word Secret Recovery Phrase must be written on paper and stored securely — it is the only way to recover the wallet.

Step 2 — Switch to the local Hardhat network. In MetaMask: network dropdown → Add a custom network. Enter: Name = Hardhat Local, RPC URL = `http://127.0.0.1:8545`, Chain ID = 31337, Currency = ETH.

Step 3 — Import a test account. In MetaMask: account icon → Import account → paste one of the private keys printed by `npm hardhat node` (Account #0, #1, or #2).

Step 4 — Start the local blockchain.

```
1 cd smart_gr_contract
2 npm hardhat node           # keep this terminal open
```

Step 5 — Deploy the contract.

```
1 # In a second terminal:  
2 npx hardhat run scripts/deploy.js --network localhost  
3 # Copy the printed contract address.
```

Step 6 — Update the frontend. Open `frontend/index.html`, find `const CONTRACT_ADDRESS` and replace the value with the address from Step 5. Save and open in Chrome/Firefox.

Step 7 — Run all tests.

```
1 npx hardhat test  
2 # Expected output: 75 passing
```